USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

WiFi, Touch, and the Column That Ate the 3D View

UM-NATOS-028

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-028	1.0 · 2026-08-16	**Receive working, transmit not, display fixed**	Internal

1. Abstract

This report covers one long day that went: *let us finish the WiFi MAC → hold on, why is `task_sleep` not sleeping → the touchscreen has gone strange → and now the 3D view is torn → ah, it was a rectangle of black paint the whole time*.

It ends well. nat-os now **receives 802.11 frames**, decodes beacons, names the networks around it, and keeps doing so continuously while rendering a 3D view. It also **transmits** — in the sense that the MAC accepts frames and reports them complete — but nothing on Earth has yet heard one, which is a distinction §5 will insist on at some length.

The debugging is the interesting part, as usual. Four separate theories about the display fault were investigated, measured, and executed. All four were wrong. The fifth was a `display_fill_rect()` that had been minding its own business for weeks and only became a problem when something else grew to 240 pixels wide.

There is a running theme, and it is not "hardware is hard". It is that **several of this kernel's own instruments were lying**, and each one sent the investigation somewhere expensive. A tally is kept in §8.

2. The WiFi arc, briefly, because it actually worked

2.1 The OSI table gets bodies

`wifi_osi_funcs_t` is Espressif's 116-entry function table: the entire contract between the MAC blob and whatever operating system is hosting it. Previously nat-os provided all 116 entries with the correct types, in the correct order, containing nothing whatsoever. It linked beautifully and would have collapsed the instant anything called it.

Thirty-nine entries now do real work: semaphores, mutexes, queues, event groups, software timers, the malloc family, delays, tick conversion and a PRNG. The remaining stubs are things the MAC has not asked for yet.

The shape of the solution matters more than the count. The table **must** be windowed ABI, because libpp calls it. The kernel **is** call0 and always will be. So every windowed entry does no work at all — it forwards through `w2c_callN()` in `window.S` into `wifi_osi_impl.c`, where the heap and scheduler actually live:

```
libpp (windowed) → wifi_osi.c entry (windowed, does nothing)
                 → w2c_callN (window.S)
                 → wifi_osi_impl.c (call0: heap, scheduler, tick)
```

Design notes worth keeping:

- **Fixed pools, not the heap.** The blob allocates during init and holds its objects forever. Static arrays cannot fragment, cannot fail late, and cost a known amount of DRAM.
- **Handles are tagged indices** (`0x05100000 | index`), checked on every use. The blob is not code anyone here can audit, and a stale handle should be rejected rather than become a wild write.
- **Blocking uses `task_sleep` , not `task_block`** — even for an infinite wait. A lost wake-up on `task_block` never runs again. This costs a tick of latency on a missed wake; the caller re-checks anyway. Deliberate pessimism.

`ositest` drives the table through its function pointers exactly as libpp will, so each check crosses windowed→call0 and back. All six pass, pools return to zero, nothing leaks.

2.2 The MAC wakes up

open-mac's entire MAC initialisation is one masked register write:

```
MAC_CTRL_REG = MAC_CTRL_REG & 0xffffe800;
```

The work was proving it did anything. This kernel has been caught **three separate times** by peripherals whose registers read back perfectly while the hardware sat completely dead — LEDC behind `DPORT_PERIP_CLK_EN` bit 11 being the clearest (UM-NATOS-027 §3.3). A readback is not evidence.

So instead: scan the 4 KB MAC register window, wait, scan again, and count words that moved with nothing driving them. A gated block is static. A running MAC has free-running counters in it, and no write that went nowhere can fake one.

stage	moving words
cold boot	**0**
after `phyinit`	**6**
after `macinit`	**13–15**

2.3 Finding the TSF timer by behaviour

Espressif publishes nothing about this register map, and every address in circulation is reverse-engineered. Rather than assert that some offset is the TSF timer, the scan reports the *rate* of every mover:

```
0x3ff73c00  1000 kHz   <- stable across every run
0x3ff73c14  1054 -> 1102 kHz
0x3ff73c18  1055 -> 1107 kHz
0x3ff73dd0  1063 -> 1114 kHz
```

`0x3ff73c00` is the only word reporting **exactly** 1000 kHz every time; everything else drifts. Confirmed over a long interval against the CPU's own cycle counter — two clocks with no connection to each other:

```
tsf advanced 622458 over 622 ms of cpu time -> 1000 kHz
tsf advanced 508686 over 508 ms of cpu time -> 1001 kHz
```

Agreement to 0.1% over half a second is not something a misread address produces. **nat-os now has a 1 MHz timebase**, which it did not have before — the kernel tick is 10 ms.

2.4 The MAC address, and a better oracle

A six-byte address looks equally plausible in any byte order, so the decode needed checking against something. The first attempt compared the top three bytes against a list of Espressif OUIs and reported **failure** for `5c:01:3b:50:3f:64`.

The decode was right. The **list** was wrong. That test was measuring a recollection of IEEE registrations, not the hardware.

eFuse burns a CRC8 of the address in the same block. Checking against that tests the decode against data on the chip: a wrong byte order cannot pass, and no outside knowledge is involved. `5c:01:3b:50:3f:64` gives `0x08`, which is the stored byte; the reversed order gives `0x8f`.

The better oracle was already on the chip. This will come up again.

2.5 It receives

```
802.11 frame: BEACON  len=348
bssid 44:25:38:19:0d:1a   ssid "TC7NR"
```

A real beacon from a real access point, on a kernel built `-mabi=call0` running Espressif's PHY blob through hand-written window handlers.

The blocker had been `chip_v7_set_chan_nomac` panicking with StoreProhibited. Three theories; the two wrong ones cost the most, and the measurements that killed them were the valuable part:

1. **Stack depth.** The PHY got a private 6 KB stack. The fault moved *forward* without going away — which looked like progress and was not. Priming the stack with a pattern and reporting its high-water mark from the panic handler settled it: **272 bytes of 6144 used**. Depth was never the problem, and without that number the obvious next move was a bigger stack, which would also have failed.
2. **Stale `WINDOWSTART` bits.** Declaring exactly one live frame before entering the blob: no change.
3. **The base frame was not a windowed frame.** This was it.

`_WindowOverflow8` — the handler in this very repository — spills `a4..a7` relative to the caller's stack pointer, which it fetches with:

```
    l32e    a0, a1, -12             /* a0 <- call[j-1]'s sp */
```

Every windowed frame must carry its caller's stack pointer at `sp - 12`. `ENTRY` does not write it; the *caller's prologue* does. `phy_stack_call` is call0 code and wrote nothing there, so the handler read a fresh `.bss` zero and spilled to `0 - 32`.

That also explains why the reported PC was never where the store was: the spill faults *inside* the overflow handler, which runs with `PS.EXCM` set, so the second fault vectors to the double-exception handler while `EPC1` still holds the instruction that caused the *original* overflow — the `call8`. Hours were spent disassembling the wrong instruction.

The fix is one store. Everything after it worked first try.

2.6 Continuous reception

Recycling descriptors per open-mac's `rs_recycle_dma_item` turned "captured four frames once" into a working scanner:

```
frames=372  recycled=374  networks=2
   44:25:38:19:0d:1a  x199  "TC7NR"
   38:88:71:2d:c1:cd   x53  "Verizon_S6QHX4"
```

With four buffers, 372 frames can only come from reuse. It also kept receiving *through* a 3D rendering session — the radio and the display share nothing but the scheduler, and neither starved.

---

3. Transmit, and the difference between "sent" and "sent"

The MAC accepts frames and reports every one complete:

```
tx handed to hardware=178  completions reaped=178   forced=0
```

This report initially described a rising completion count as "the MAC saying the frame actually went out". That was an overclaim, and the user checked their phone for the beacon SSID, and it was not there.

The test that settles it is a **probe request**. A beacon is a statement and nothing has to answer it, so silence proves nothing. A broadcast probe request with a wildcard SSID obliges every access point in range to send a probe *response* addressed to this station — and one arriving is unforgeable, because a radio cannot hear itself and nobody sends to this MAC without having heard from it first.

```
sent 20 probe requests
completions reaped=20
frames addressed to us=0
```

Twenty frames the hardware called complete, zero answers from two access points loud enough that we receive them continuously. **A completion bit says the frame left the queue, not that it left the antenna.**

Two candidate causes were eliminated by measurement:

- **Transmit power.** `most_tpw` already reads `0x28` — 40 quarter-dBm, **10 dBm** — straight out of `register_chipv7_phy`. Never zero. `phy_set_most_tpw(78)` returns success and changes nothing.
- **The MAC hardware init chain.** `ic_mac_init`, `hal_init`, `ic_enable_rx` and `hal_mac_tsf_reset` all run without a crash. Still zero answers.

That last one produced two genuinely good pieces of news:

The IRAM panic was unfounded. Referencing those four functions cost **2,459 bytes** (116,692 → 119,151 text against 131,072 of IRAM), not the 48 KB this project has warned about since `MAC-NEXT.md`. That figure came from a `--whole-archive` measurement, which pulls every object; a real link pulls only what is referenced. A constraint that had been shaping decisions for weeks simply did not exist.

The blob runs. This was the first time nat-os executed libpp at all, and the OSI table, the window handlers and the mixed-ABI bridges all held up under the code they were built for. That was never certain.

Also learned, expensively: **naming a symbol changes the link.** Referencing `ram_tx_pwctrl_bg_init`, which was not previously linked, pulled fresh objects out of `libphy.a`, after which `register_chipv7_phy` died with IllegalInstruction inside `set_rx_gain_testchip_70` — a calibration function nobody had touched and which had worked for days. Rule adopted: **reference only what open-mac references**, and check with `nm` that a symbol is already in the image before calling it.

Strongest untried lead: `periph_module_reset(0x19)`. nat-os has only ever *ungated* the WiFi peripheral, never *reset* it, and a MAC left in whatever state the ROM bootloader put it in would plausibly receive while refusing to transmit. That asymmetry fits the symptom exactly.

---

4. The `task_sleep` detour, which was not a detour

Mid-WiFi, the TSF check reported ~1 µs for a requested 500 ms sleep. Two independent clocks agreed on that, so it was not a measurement artefact.

Two bugs, stacked.

`timer_ticks()` counted ISR entries, not time. `task_yield()` ends a slice by pulling `CCOMPARE1` back to `ccount + 64`, so `timer_isr` ran on every yield as well as every real deadline — and `g_ticks++` sat at the bottom of it unconditionally. Measured at **217 ticks per real second** where 100 is correct, and far higher when a task sat in an idle-yield loop. Every deadline expressed in ticks came due early, in proportion to how much yielding the system happened to be doing.

`task_yield()` arms a switch; it does not perform one. It writes the comparator and returns, so the caller keeps running for ~60 cycles into what it believes is a sleep. `task_sleep(50)` — half a second — returned to its caller in **107 cycles**, which is the cost of the function body and nothing else. The sleep did happen; it started after the caller had already read the clock and concluded no time had passed.

Both fixed. `task_sleep(50)` now takes 639 ms / 64 ticks, and 64 × 10 ms = 640 ms — the tick count and the wall clock agree, which they did not before.

This mattered for WiFi specifically: every OSI queue and semaphore timeout is built on `task_sleep`, so anything the blob did with a timeout was quietly unreliable. But the wider point is the uncomfortable one, and `timer.c` had already written it down about an *earlier* bug in the same function: **nothing showed a symptom.** Sleeps ran short, timeouts ran loose, and every frame-rate figure this project has ever recorded was measured against a clock running fast by a load-dependent factor.

---

5. The touchscreen gets worse, twice

5.1 First regression: a background task outranking the user

The WiFi receive task was raised to `HIGH` to get beacons from 3 Hz to 9.4 Hz. The raycaster blit was checked, found unchanged, and the change declared a clean win.

The touch task is `NORMAL`. With **two** flat-out `HIGH` tasks instead of one it stopped being scheduled except when ageing rescued it — roughly every 300 ms. The panel went from responsive to intermittent, which no counter in this kernel reports and which the user noticed within minutes.

It was also a bad trade on the merits: beacon timing was prioritised over input latency, and the beacons were not reaching the air anyway, so the 9.4 Hz was entirely theoretical.

The useful measurement here was the one that ruled the obvious suspect **out**. `update_rx_chain()` spins on a hardware acknowledge and looked exactly like the cost; instrumenting it showed a worst-case wait of **one** iteration. It was never the radio, it was the scheduler.

5.2 Second regression: a faithful restoration that was not

Reverting the priority was not enough — touch degraded to needing a press-and-hold. The cause was an *earlier* change that had been described, in a commit message, as a "restoration".

The touch loop originally called `task_sleep(1u)` back when `task_sleep` neither slept nor yielded. So it polled **repeatedly inside whatever slice it got**, which is exactly why touch felt continuous. Replacing it with `task_yield()` looked equivalent and was not: a yield surrenders the CPU after **every single poll**, so the task sampled once per slice instead of many times, and a quick tap fell between samples.

Fixing `task_sleep` had, with perfect irony, broken the thing that was accidentally relying on it being broken.

The final configuration is a real 10 ms sleep at `HIGH` priority — 100 Hz sampling for one SPI read per tick, which is far less CPU than the old flat-out polling and, unlike it, has a rate that is a property of the clock rather than of whatever else happens to be running.

	before	after
touch samples per reporter interval	~15	**~150**

6. The 3D view, and four confident wrong answers

Then the 3D view started tearing, and the close button vanished.

6.1 Theory one: DMA had fallen back to the FIFO path

The DMA wait is bounded on **wall clock** at ~25 ms, justified in a comment as "far beyond the ~100 µs a 480-byte transfer needs" — true of the *transfer* and false of the *wait*, because a preempted task can exceed it easily. And a timeout **permanently** disables DMA. The reasoning was sound.

The counter said `dma=320/0`. Zero timeouts. Ever.

The bound was raised anyway on the general principle that a wall-clock wait should survive preemption, then restored when it fixed nothing.

6.2 Theory two: touch preempting the display mid-transfer

Instead of one flash per guess, both knobs were made runtime-tunable (`touchcfg <prio> <sleep>`), and all four combinations tested in a single build:

```
NORMAL + yield   31399 31332 31394
HIGH   + yield   31352 31398 31353
NORMAL + sleep1  31399 31346 31385
HIGH   + sleep1  31424 31387 31438
```

Identical. Touch scheduling does not move it. **This is the single best thing that happened during the display investigation**, and it should have happened three rounds earlier.

6.3 Theory three: applications drawing over the view

`ps` showed `ping` and `pong` running and drawing, with `drawskip` climbing — proof that some draws were being skipped, and therefore that others were succeeding. Killed all three applications. `drawskip` froze. No change on screen.

6.4 Theory four: the panel clock

The display driver has documented since UM-NATOS-015 that clocking this panel too fast puts visible noise on the glass while **every counter reports success**. That is a suspiciously exact description of the symptom, so a runtime clock selector was added (`spiclk 0|1|2`) and the panel dropped from 40 MHz to 10 MHz.

Different. Still wrong.

At this point the investigation had a genuinely misleading piece of evidence: the same commit rendered correctly earlier in the day and torn later, with no code change in between. That reads as hardware. A reasonable-sounding case was made for a marginal display flex and a sagging USB rail, and the user was asked to reseat connectors.

The user replied, in effect, *no, it is a bug in the code, because the X button is also missing*.

They were right. What differed between the two runs was not the hardware. It was which application slots happened to be occupied — see §6.6.

6.5 The test that should have been first

A **static test pattern** through the raycaster's exact blit call: same buffer, same width, same stride, same contiguous path. Six vertical colour bars, plus a white block in the top-right corner where the close button goes.

This splits one question into two, and the two have completely different answers:

- If the pattern is sheared or torn → the **transport** is broken.
- If the pattern is clean but something is missing → the content is fine and something **overwrites it afterwards**.

Result: **clean bars, and a hole exactly where the white block had been written.**

(The first attempt at this panicked, because `raycast_framebuffer()` returns a *boolean* and the test used it as a pointer, storing through address `0x00000001` . `raycast_fb_ptr()` now exists so nobody repeats that. It was a fast and educational panic.)

6.6 The actual bug

`desktop_chrome()` repaints a right-hand column every frame, one strip per application slot — a name and a red X for running slots, and **solid black** for empty ones:

```
display_fill_rect(CHROME_X, y, APP_CHROME_W, APP_VIEW_H, COLOR_BLACK);
```

That column is documented as reserved *outside every application viewport*, which is entirely true — for applications, whose viewports are narrower.

The 3D view is not an application. `RAY_VIEW_W` is 240. `DISP_W` is 240. **The view owns the whole width, including that column**, and the chrome runs immediately after `raycast_frame()`, painting over the right edge of a view that had just drawn it.

One cause, every symptom:

- the tearing along the right edge
- the missing close button
- the vanished white test block, in precisely that corner
- and why killing the applications made it **worse** — an empty slot fills the column with solid black, while a running one at least draws text

That last point is also why "the same binary behaved differently" was so convincing and so wrong. The binary was identical. The application slots were not.

The fix follows a pattern already established in that file for exactly this reason: a full-width view stamps its own close button into the framebuffer so it arrives in the same transfer, and the chrome loop now returns early in `MODE_3D`. Applications remain closable from the launcher.

---

7. Where things ended up

3D view	fixed, close button present, `corrupt=0`
Touch	100 Hz at HIGH priority, ~10× the previous sample rate, default
WiFi receive	working — continuous scanning, beacon decode, descriptor recycling
WiFi transmit	MAC accepts and completes frames; nothing hears them
`task_sleep`	actually sleeps, for the first time
`timer_ticks()`	actually counts time, for the first time
IRAM budget	not a real constraint; the 48 KB figure was a measurement error

---

8. The instruments that lied

This is the part worth keeping. Every one of these cost real time.

1. The blit timer. `t_blit` uses `task_cpu_cycles()`, which only advances when the scheduler credits a slice at a context switch — it does not tick *inside* one. So the figure tracks context switches, not work. A 55.85 → 31.3 ms shift was read as a serious regression, reported to the user as a 44% *improvement* an hour earlier, and was neither. It was an artifact both times.

2. `timer_ticks()`. Counted ISR entries rather than elapsed time, so every tick-denominated deadline in the kernel came due early by a load-dependent factor. Silent for months.

3. The transmit completion bit. Reports that a frame left the queue. Was described as proof the frame left the antenna. It is not, and a phone was the instrument that caught it.

4. The OUI list. A hand-written table of Espressif prefixes declared a correct MAC decode invalid. The chip's own CRC8 was sitting right there.

5. The PHY stack high-water mark. Reported `6144 of 6144 bytes used` in a real panic report — not a large number, but a *missing measurement*, because an unprimed `.bss` stack is all zeros and the scan reads that as fully consumed. It looked exactly like a stack exhaustion that had not happened.

6. `raycast_framebuffer()`. Returns a boolean. Named like an accessor. Panicked a diagnostic that trusted the name.

7. `fb=on` in the reporter. Reports the framebuffer *mode*, not whether a buffer exists — fine, but easy to misread while hunting a rendering fault.

The pattern across all seven: **the counters were all clean while the picture was visibly wrong.** `corrupt=0`, `dma=320/0`, no timeouts, no skew, no lock contention worth mentioning. Every automated check this kernel has said everything was fine, and every one of them was telling the truth about the narrow thing it measured.

The person looking at the screen was the only instrument that could see the actual fault, and twice in this session that person was right while the counters and the reasoning were wrong. Worth remembering before the next confident hardware diagnosis.

---

9. Method notes for next time

Three things worked well enough to be worth repeating deliberately:

Make the variable runtime-tunable before forming a theory about it. `touchcfg` turned four hypotheses into one flash. `spiclk` did the same for the panel clock. Both were written *after* several reflash-per-guess rounds that they would have eliminated.

When something looks corrupted, draw a static test pattern first. It separates "the content is wrong" from "something overwrites it afterwards" in a single step, and those two have disjoint suspect lists. This was the fifth thing tried and should have been the first.

Prefer an oracle that lives on the chip. The eFuse CRC beat a hand-maintained OUI list. Two independent clocks agreeing beat one clock asserting. A hardware acknowledge bit clearing beat a register reading back what was written.

And one process note: when a user says *it worked a minute ago on this exact build*, that is data, but it is not automatically evidence of hardware. It is equally evidence that **something in the run-time state differs** — in this case, which application slots were full. Enumerate that state before reaching for the screwdriver.

10. Open: the startup glitch, and a very good clue

Not everything got solved, and the unsolved part has the sharpest clue in the whole report.

Symptom. Opening the 3D view sometimes shows a wrong picture for ten to thirty seconds, then it comes good on its own. A second program (`gfxrogue`) was observed glitching over the same period and recovering at the same moment.

What is measured and known:

- With **no applications running**, the view is correct **immediately**. That is a clean A/B and rules out the renderer in isolation.
- In steady state there is no contention to speak of: 18-19 frames per reporter interval, `cont=0` , `dlock blk=0` , with or without applications.
- There is no periodic stall. A 55-second capture shows `dHold` steady at ~2000 ms per interval and frames advancing uniformly.
- The camera is not burying itself in a wall — it wanders normally from the first sample.
- **Opening `gfxrogue` makes the 3D view correct.** Starting another program REPAIRS it.

That last one is the interesting one. If a second program starting fixes the first, the view is missing an initialisation step that an application start happens to perform, rather than being actively corrupted by anything.

A wrong turn worth recording. The first fix returned from `desktop_chrome()` outright in `MODE_3D` , reasoning that the chrome column was painting over a full-width view. The geometry disagrees: `APP_VIEW_Y0` is 224 and `RAY_VIEW_H` is 224, so the strips begin exactly where the view ends. The guard has been narrowed to a `_Static_assert` that fails the build if the view ever grows into the strips — which is the check that should have been written first, since it answers the overlap question at compile time instead of from the glass.

The blittest that produced that theory was also contaminated: it called `desktop_set_active(1)` to stop the renderer overwriting the pattern, which put the launcher into repaint mode and erased the test block itself. **A diagnostic that changes the state it is measuring is worse than no diagnostic**, and this one was believed for several rounds.

Everything ruled out so far, each by direct measurement rather than argument:

theory	how it died
DMA fell back to the FIFO path	`dma=320/0` — the guard has never fired
Touch scheduling	identical blit across all four priority/poll combinations
Applications overdrawing the view	killed them all; `drawskip` froze; no change
Panel signal integrity	clock 40 -> 10 MHz changes appearance, does not fix
The chrome column	`APP_VIEW_Y0` 224 == `RAY_VIEW_H` 224; they never overlap
Movement catch-up burst	fixed by `raycast_open()` ; symptom unchanged
Touch calibration	was genuinely broken and is fixed; symptom unchanged
An arena overlapping the framebuffer	`fb 0x3ffbcd70..0x3ffd7170` , arenas from `0x3ffd7590` — clear

What is known for certain:

- The renderer runs at FULL RATE throughout. A 54-second capture of a real tap-driven session shows `act=0` and frames climbing steadily at ~8 fps for the entire bad period. This is not slowness.
- The framebuffer holds a real rendered scene, and the camera wanders normally rather than being stuck in geometry.
- Transport is clean: a static colour-bar pattern renders correctly through the raycaster's own blit call.
- The picture is **garbled**, not frozen, not blank.
- Starting a program REPAIRS it, without leaving the view.

That last fact is the whole remaining mystery and no mechanism for it has been found. `app_start()` allocates an arena, copies an image, initialises a VM, sets a viewport and clears IPC. None of that touches the panel, the framebuffer or the display driver.

Next step for a fresh session: instrument `app_start()` itself — take an `fbsum` immediately before and immediately after a program launch while the view is visibly bad, and see which of those five operations changes the buffer. That converts the one unexplained fact into a measurement instead of a hypothesis, which is the move that has worked every other time in this report and was reached for too late each time.

Filed under: things that were not the WiFi's fault.